

LAPORAN TUGAS

STURKTUR DATA DAN ALGORITMA

PENJELASAN TAMBAHAN TUGAS 4 BST



Dosen Pengampu:
Prof. Dr. Taufik Fuadi Abidin, M.Tech

Disusun oleh:
Muhammad Aqil Mubarak (250810701100003)

JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
BANDA ACEH

2026

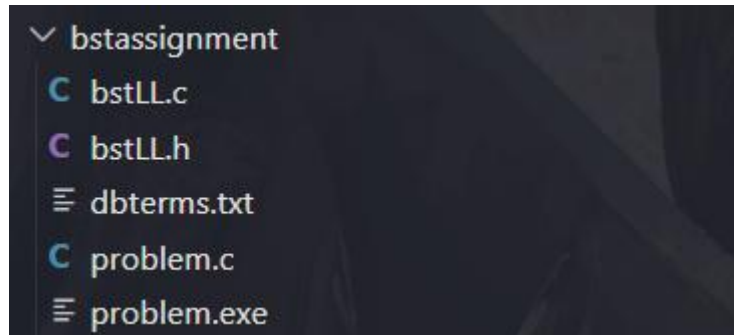
PROBLEM TUGAS 4

Problem

Merancang program dan membangun gabungan struktur data Binary Search Tree dan Linked List untuk menghimpun kata-kata yang ada dalam sebuah file teks berdasarkan huruf awal dari kata tersebut. Anda dimintakan untuk merancang sebuah program yang menghimpun kata-kata yang diperoleh dari sebuah file ASCII (format teks) berdasarkan huruf awal dari kata tersebut ke dalam struktur data Binary Search Tree (BST) dan Linked List. Simpul (node) BST terdiri dari sebuah huruf, pointer head ke struktur data Linked List, jumlah data dalam Linked List, dan pointer left dan right. Jika huruf dalam sebuah node BST adalah 'a' maka kata-kata yang ada dalam Linked List yang ditunjuk oleh pointer head pada node BST adalah kata yang diawali dengan huruf 'a'. Jika huruf dalam node BST adalah 'b' maka kata-kata yang ada dalam Linked List adalah kata-kata yang diawali dengan huruf 'b', begitu seterusnya untuk setiap node BST. Node Linked List terdiri dari sebuah kata dan baris dimana kata itu ditemukan dalam file. Program menerima input berupa file teks yang pada setiap barisnya terdapat sebuah kata. Kata-kata dalam file teks tersebut berjumlah 25988 (ada duplikasi) dan disusun secara tidak teratur. Program harus menyediakan pilihan untuk menampilkan data tertentu dalam BST dan linked list, menampilkan statistik dari kata secara keseluruhan dan teratur, dan menghapus sumber daya memori yang telah digunakan (*free memori without memory leak*).

PENJELASAN TAMBAHAN

Dalam Pengerjaan solusi problem ini, saya menggunakan pendekatan modularitas, dimana program dipecah menjadi beberapa file terpisah yaitu file header dan file implementasi. Cara ini memisahkan logika dasar struktur data dengan logika utama penyelesaian pada problem, sehingga kode menjadi rapi dan mudah di kelola.



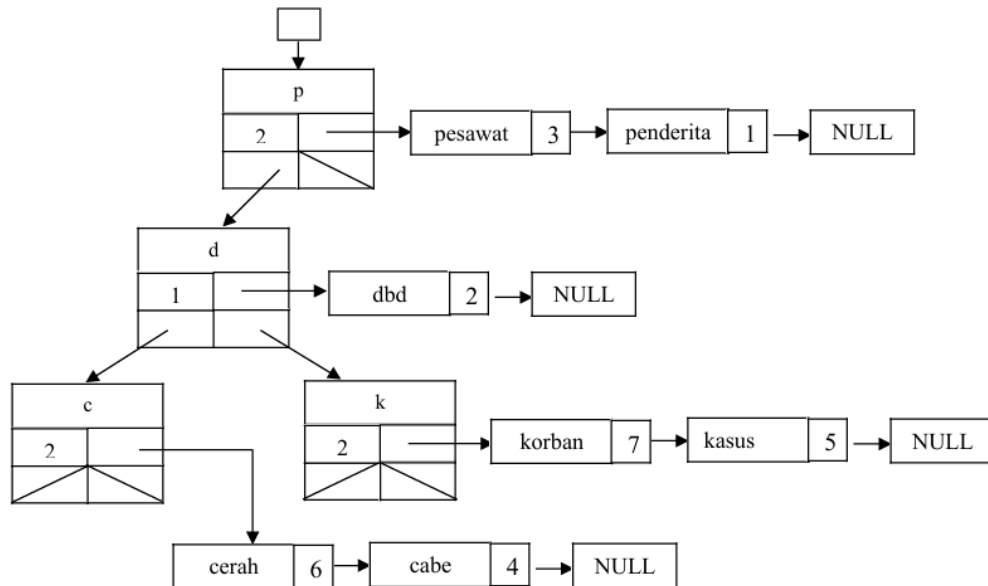
Gambar 1. Direktori File

Program dapat di compile dengan command:

```
'gcc -Wall -pedantic -g -o problem problem.c bstLL.c '
```

Problem

Penyelesaian masalah ini menggunakan kombinasi dua buah struktur data dinamis, yaitu *Binary Search Tree* (BST) dan *Linked List*. BST dipilih sebagai struktur utama karena kemampuannya dalam melakukan pencarian (*searching*) dan penyisipan (*insertion*) data secara efisien dengan kompleksitas waktu logaritmik, yang sangat ideal untuk memetakan alfabet 'a' hingga 'z'. Sementara itu, *Linked List* digunakan sebagai struktur data sekunder di dalam BST untuk menangani terjadinya kolisi data (kondisi di mana terdapat banyak kata yang berawalan huruf yang sama). Dengan pendekatan ini, pengelompokan 25.988 kata dapat dilakukan secara efisien tanpa harus mendeklarasikan *array* statis yang boros memori.



Gambar 2. Gambaran Tree Linked List

Gambar di atas mengilustrasikan arsitektur utama dari struktur data yang dibangun. Sebuah akar (*root*) dari pohon akan membagi simpul-simpul huruf ke cabang kiri jika secara alfabet lebih kecil, dan ke cabang kanan jika lebih besar. Perlu ditekankan bahwa setiap simpul (*node*) pada BST tidak menampung teks kata secara langsung, melainkan bertindak sebagai penunjuk (*head pointer*) menuju sebuah *Linked List*. Sebagai contoh, pada simpul huruf 'k', terdapat *Linked List* yang merangkai setiap node kata yang berawalan huruf 'k' secara sekuensial (seperti 'korban', 'kasus', dan 'kampung'), di mana setiap node pada *Linked List* tersebut juga menyimpan informasi metadata berupa nomor baris posisi kata tersebut di dalam file teks.

```

1  typedef struct list {
2      NodePtr head;
3      unsigned size;
4  } List;
5
6  // Struct BST
7  typedef struct intbstNode * instbstNodePtr;
8  typedef struct intbstNode {
9      char kata;
10     List list;
11     instbstNodePtr left, right;
12 } intbstNode;

```

Gambar 3. Code Gambaran pada Tree Linked List

Secara implementasi kode, arsitektur pada gambar sebelumnya diterjemahkan ke dalam dua buah *struct* utama. Pertama adalah *struct node* yang berfungsi sebagai entitas *Linked List*, berisi alokasi dinamis untuk teks kata (*char *kata*), penyimpanan nomor baris (*int nomor*), dan penunjuk ke data berikutnya (*NodePtr next*). Keduanya adalah *struct intbstNode* yang menjadi kerangka BST, bekerja menyimpan satu karakter alfabet sebagai kunci pencarian (*char kata*), menyimpan struktur basis *Linked List* (*List list*), serta penunjuk cabang child (*left* dan *right*). Adapun alur kerjanya adalah: saat program membaca satu baris kata, program akan mengambil huruf pertamanya, menelusuri BST hingga menemukan simpul huruf yang cocok atau membuat simpul baru jika belum ada, lalu memanggil fungsi penyisipan untuk menambahkan teks kata tersebut ke dalam *Linked List* milik simpul tersebut.



```

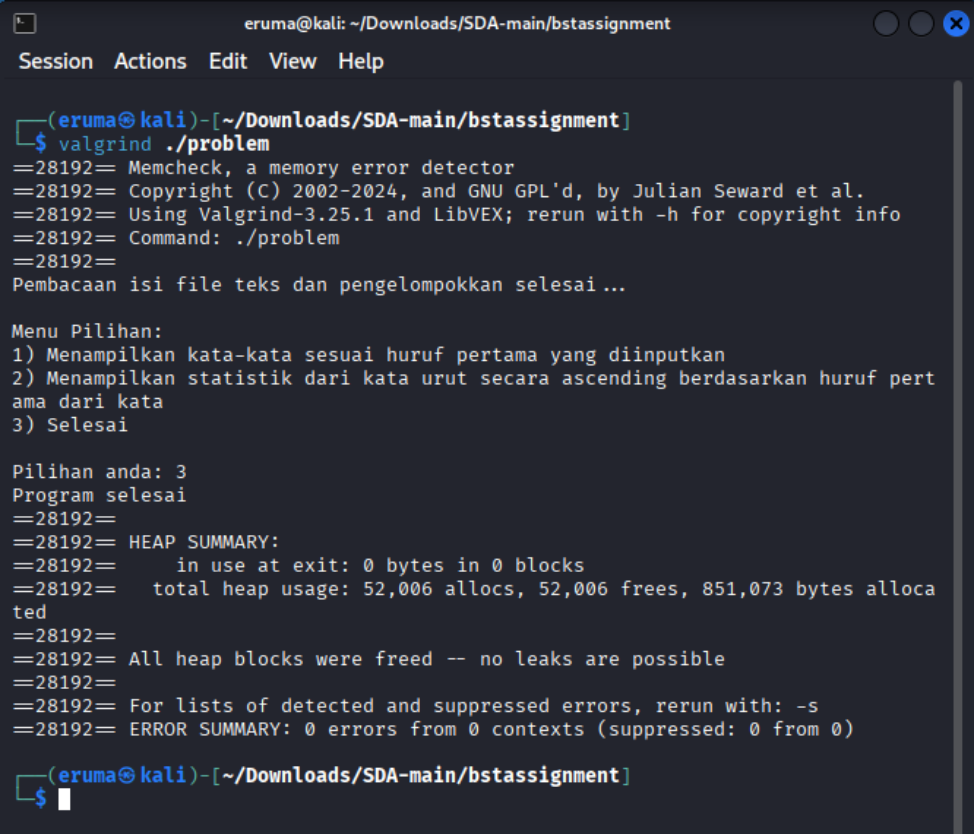
1 // Fungsi untuk menelusuri tree secara In-Order (Kiri - Cetak - Kanan)
2 void inOrder (intbstree * pBst) {
3     int top = -1;
4     instbstNodePtr stack[100];
5     instbstNodePtr curr = pBst->root;
6
7     // Jika pohon kosong, langsung keluar
8     if (pBst->root == NULL) return;
9
10    // Looping selama masih ada simpul yang dikunjungi atau stack belum kosong
11    while (curr != NULL || top != -1) {
12        // Telusuri terus cabang kiri sampai NULL
13        while (curr != NULL) {
14            stack[++top] = curr;
15            curr = curr->left;
16        }
17        curr = stack[top--];
18        printf("%c  %d\n", curr->kata, curr->list.size);
19        curr = curr->right;
20    }
21 }

```

Gambar 4. Fungsi BST Transversal inOrder

Pada Gambar 4, implementasi penelusuran *Tree* menggunakan metode *In-Order Traversal* (Kiri - Cetak - Kanan). Penelusuran ini bertugas mencetak urutan statistik kata secara alfabet (A-Z) sesuai dengan sifat bawaan *Binary Search Tree*. Implementasi pada program ini menggunakan pendekatan iteratif dengan bantuan struktur data *Stack* buatan berupa *array of pointer* berukuran 100. Keputusan ini diambil untuk menghindari beban *call stack* pada memori sistem (mencegah *stack overflow*). Kapasitas *stack* sebesar 100 sangat aman dan dijamin tidak akan mengalami *out-of-bounds*, karena kedalaman maksimal pohon (jumlah node) pada kasus ini hanya berkisar 26 simpul yang mewakili 26 huruf alfabet.

Kemudian proses pengecekan memori, memastikan tidak ada sisa memori yang bocor saat program selesai di jalankan. Pada screenshot dibawah program mencetak status “All heap blocks were freed – no leaks are possible”, yang mengonfirmasi bahwa alokasi memori dinamis (heap) telah sukses.



```
eruma@kali: ~/Downloads/SDA-main/bstassignment
Session Actions Edit View Help

(eruma@kali)-[~/Downloads/SDA-main/bstassignment]
$ valgrind ./problem
==28192== Memcheck, a memory error detector
==28192== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==28192== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==28192== Command: ./problem
==28192==
Pembacaan isi file teks dan pengelompokkan selesai...

Menu Pilihan:
1) Menampilkan kata-kata sesuai huruf pertama yang diinputkan
2) Menampilkan statistik dari kata urut secara ascending berdasarkan huruf pertama dari kata
3) Selesai

Pilihan anda: 3
Program selesai
==28192==
==28192== HEAP SUMMARY:
==28192==   in use at exit: 0 bytes in 0 blocks
==28192==   total heap usage: 52,006 allocs, 52,006 frees, 851,073 bytes allocated
==28192==
==28192== All heap blocks were freed -- no leaks are possible
==28192==
==28192== For lists of detected and suppressed errors, rerun with: -s
==28192== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

(eruma@kali)-[~/Downloads/SDA-main/bstassignment]
$
```

Gambar 5. Memory Check Problem

Gambar 5 menunjukkan hasil validasi alokasi memori menggunakan *tool* Valgrind. Pembersihan dilakukan secara *bottom-up* (dari *leaf* ke *root*) menggunakan fungsi rekursif *freeNode*. Untuk setiap simpul BST yang dikunjungi, program akan terlebih dahulu menelusuri dan melakukan *free* pada setiap elemen *Linked List* yang ada di dalamnya (*freeList*). Setelah *Linked List* bersih, barulah simpul BST tersebut didealokasi. Alur ini terbukti berhasil mencegah terjadinya *memory leak* (kebocoran memori), yang ditandai dengan pesan '*All heap blocks were freed -- no leaks are possible*' pada log Valgrind.

Output Problem:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
> .\problem
Pembacaan isi file teks dan pengelompokkan selesai...

Menu Pilihan:
1) Menampilkan kata-kata sesuai huruf pertama yang diinputkan
2) Menampilkan statistik dari kata urut secara ascending berdasarkan huruf pertama dari kata
3) Selesai

Pilihan anda: 1
Huruf: a
a abda:25986, anshari:25956, atausefalitis:25954, autocard:25944, agustiyanti:25938 (1374)

Menu Pilihan:
1) Menampilkan kata-kata sesuai huruf pertama yang diinputkan
2) Menampilkan statistik dari kata urut secara ascending berdasarkan huruf pertama dari kata
3) Selesai

Pilihan anda: |
```

Gambar 6. Output Pilihan 1

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
> .\problem

Menu Pilihan:
1) Menampilkan kata-kata sesuai huruf pertama yang diinputkan
2) Menampilkan statistik dari kata urut secara ascending berdasarkan huruf pertama dari kata
3) Selesai

Pilihan anda: 2
a 1374
b 1891
c 804
d 2244
e 445
f 410
g 566
h 639
i 619
j 502
k 2191
l 758
m 3252
n 673
o 241
p 2894
q 13
r 946
s 2682
t 1694
u 285
v 159
w 404
x 20
y 194
z 88
```

Gambar 7. Output Pilihan 2

KESIMPULAN

Kombinasi BST dan Linked List memberikan efisiensi yang sangat optimal. Pencarian huruf awal untuk menentukan kelompok kata bekerja dalam kompleksitas waktu rata-rata (*average case*) dengan kompleksitas $O(\log n)$. Meskipun pada skenario terburuk (*worst case*) sebuah BST standar dapat menurun menjadi $O(n)$ ataupun dapat sebanding dengan tinggi *tree* yaitu $O(h)$ akibat kemiringan simpul (*skewed*), performa program ini tetap terjamin sangat cepat. Hal ini dikarenakan ukuran maksimal node BST dibatasi pada sebuah konstanta abjad alfabet yang maksimal 26 node, sehingga pencarian kelompok huruf dapat dianggap hampir konstan. Setelah simpul huruf ditemukan, penambahan kata baru beroperasi seketika dalam waktu konstan $O(1)$ karena penyisipan selalu dilakukan di posisi *head* pada *Linked List* terkait. Skema ini mencegah beban iterasi yang berat dibandingkan jika harus menelusuri atau menggeser elemen pada *array* berkapasitas puluhan ribu kata.